

INTERNATIONAL INSTITUTE OF
INFORMATION TECHNOLOGY

H Y D E R A B A D



CodeGen: Fine-Tuning Small Code Models for Software Tasks

Applying parameter-efficient fine-tuning and retrieval-augmented generation to real-world developer workflows



Team Members(Group 2)

N Anjan Karthik Chandra

Jakka Rithusravya

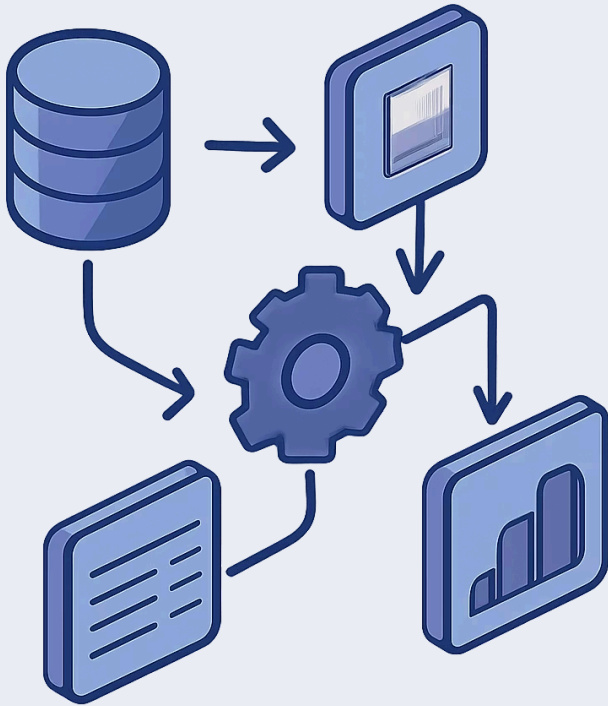
Darshna Kumawat



Mentor

Sajid Ansari

Project Overview



Target Model

Qwen2.5-Coder-0.5B — a compact code LM evaluated on three core developer tasks

Three Tasks

Text-to-SQL, documentation generation, and program synthesis

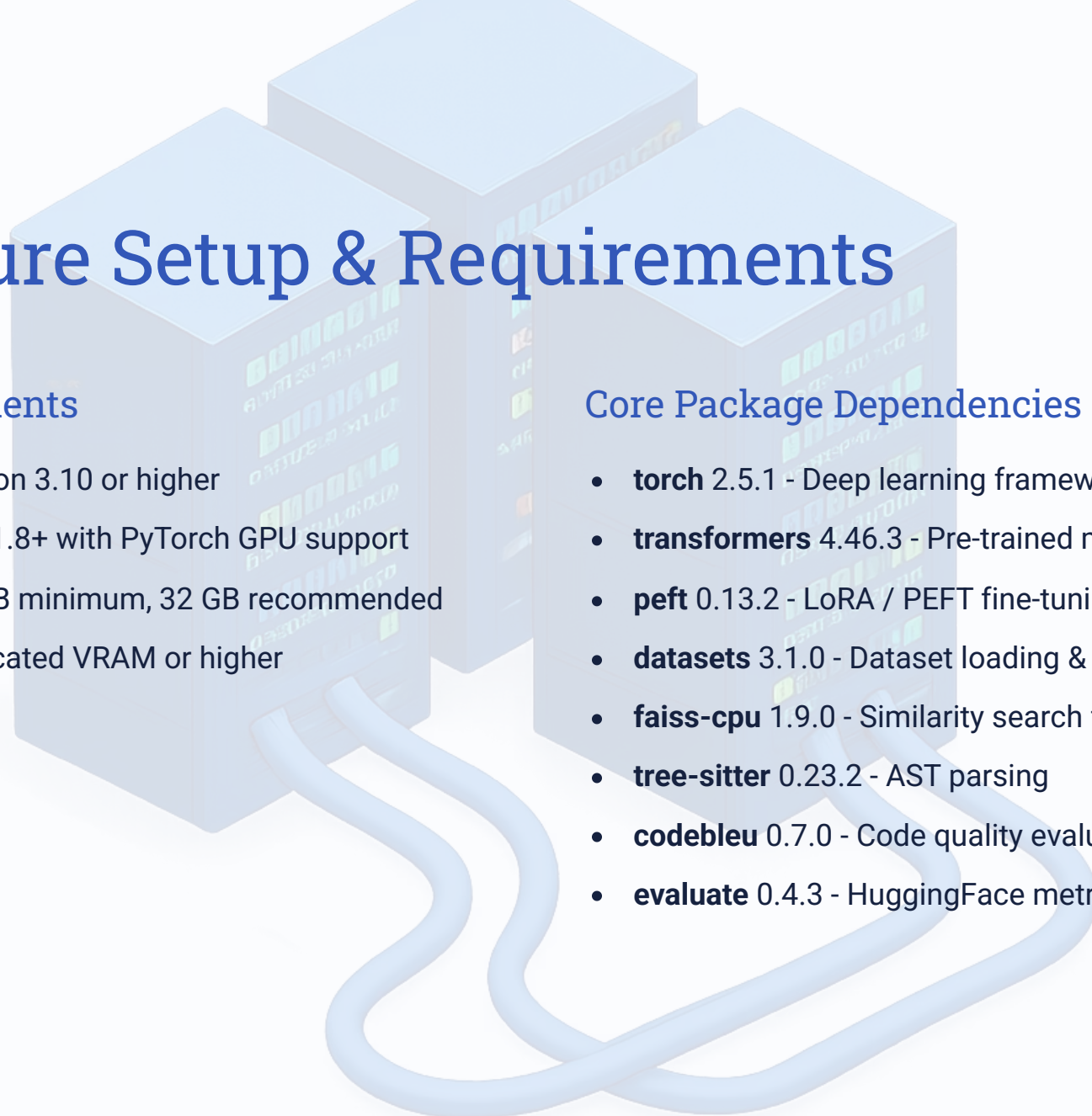
Method

LoRA fine-tuning combined with Retrieval-Augmented Generation (RAG)

Evaluation

Benchmarked against a larger LLM baseline using standard NLP and code metrics

Architecture Setup & Requirements

The background features a stylized illustration of three server racks. Each rack is light blue with a darker blue top. The front panels of the racks display various green and yellow indicator lights and some illegible text. Two thick, blue, wavy cables are shown at the bottom, looping around the base of the server racks.

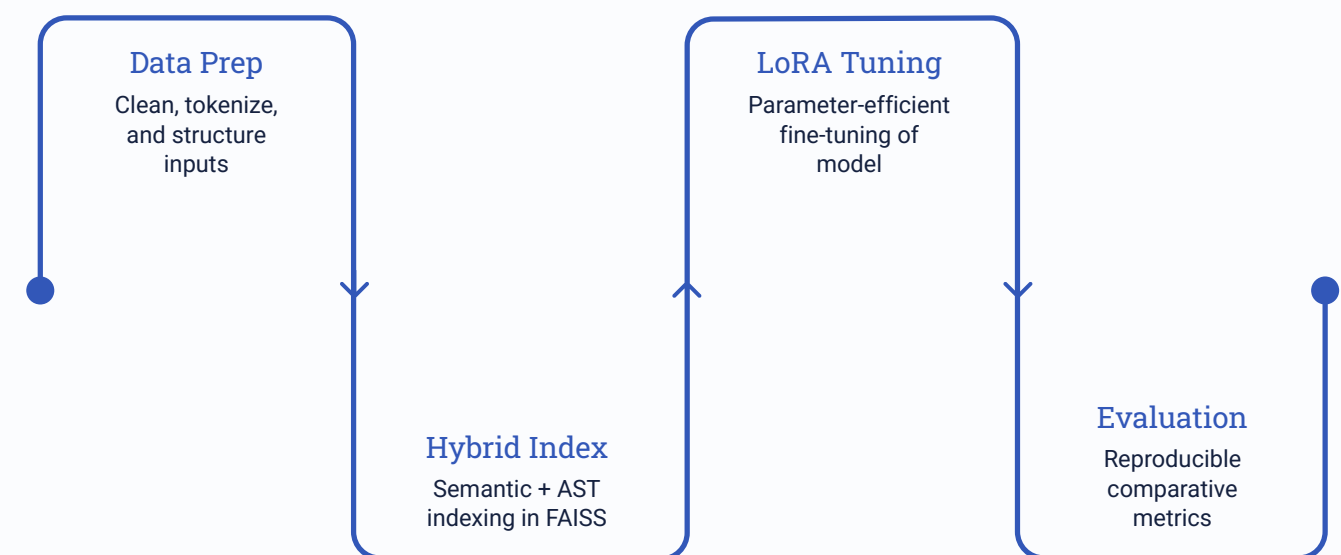
Hardware Requirements

- **Python Runtime:** Version 3.10 or higher
- **CUDA Toolkit:** CUDA 11.8+ with PyTorch GPU support
- **System Memory:** 16 GB minimum, 32 GB recommended
- **GPU VRAM:** 8 GB dedicated VRAM or higher

Core Package Dependencies

- **torch** 2.5.1 - Deep learning framework
- **transformers** 4.46.3 - Pre-trained model hub
- **peft** 0.13.2 - LoRA / PEFT fine-tuning
- **datasets** 3.1.0 - Dataset loading & management
- **faiss-cpu** 1.9.0 - Similarity search for RAG
- **tree-sitter** 0.23.2 - AST parsing
- **codebleu** 0.7.0 - Code quality evaluation
- **evaluate** 0.4.3 - HuggingFace metrics

System Architecture



End-to-end pipeline enabling reproducible, comparative evaluation across all model variants.

Core Components

- Hybrid Retrieval**
Semantic + AST/structural indexing via FAISS vector store
- LoRA Adapter**
Parameter-efficient fine-tuning on attention projection layers
- Comparative Evaluation**
Base model · Fine-tuned · RAG pipeline · LLM baseline

End-to-End Pipeline Architecture

Phase 1: Environment & System Setup

- Load Configuration (YAML, GPU, System Paths)
- Load Datasets (Spider)
- Load Base Model (Qwen2.5-Coder-0.5B Tokenizer & Weights)

Phase 2: Core Task Pipelines

- Program Generation (BLEU, CodeBLEU, Pass@k)
- Documentation Generation (BLEU, BERTScore)
- Text-to-SQL Generation (Execution Acc, Exact Match)
- Commit Message Generation (BLEU, ROUGE)

Phase 3: Semantic Indexing & RAG Engine

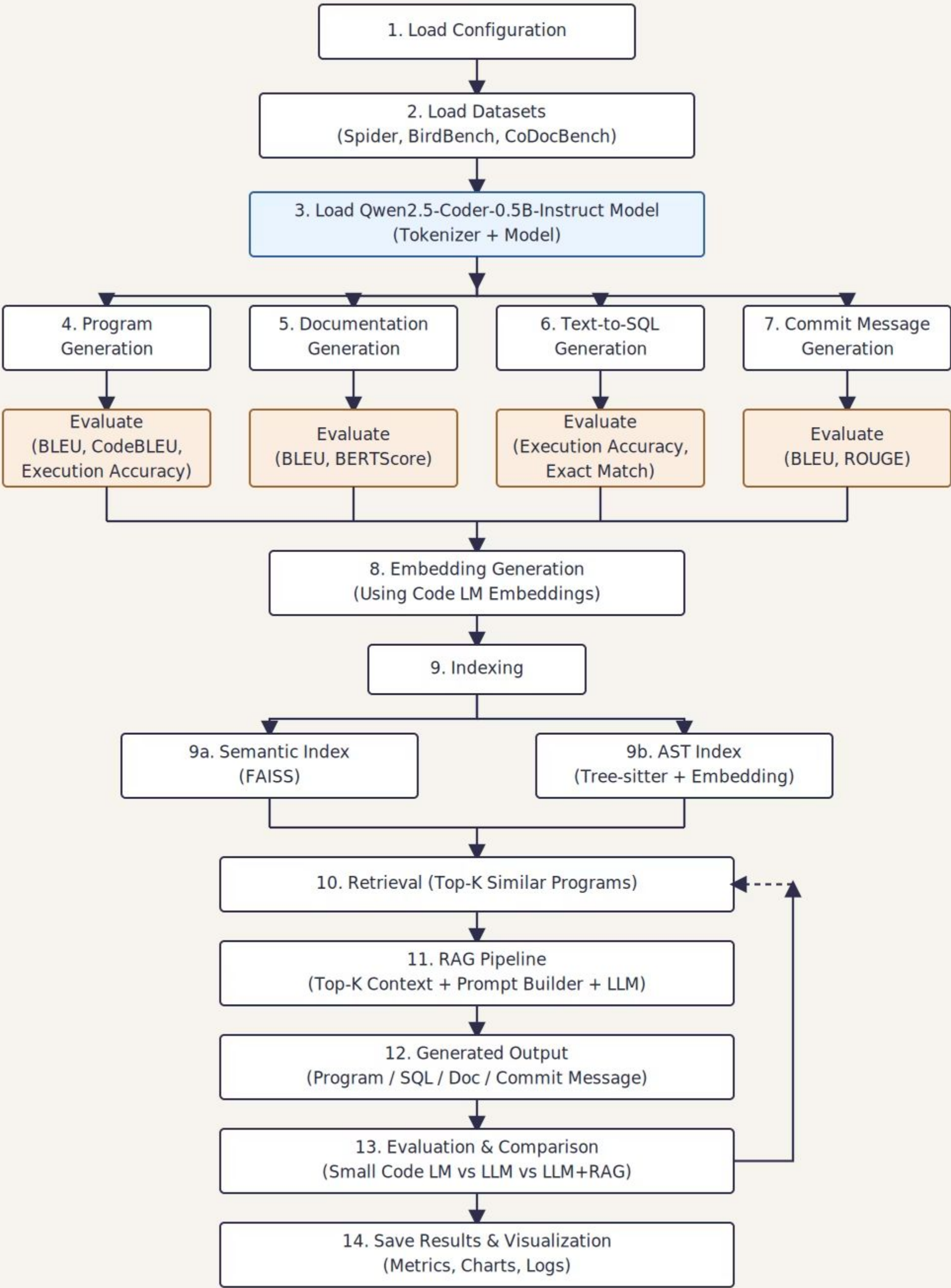
- Embedding Generation (Code LM Vector Embeddings)
- Dual Indexing: FAISS (Vector) + Tree-sitter (AST)
- Top-K Retrieval Engine
- Context Injection Pipeline (Prompt Builder + LLM Aug.)

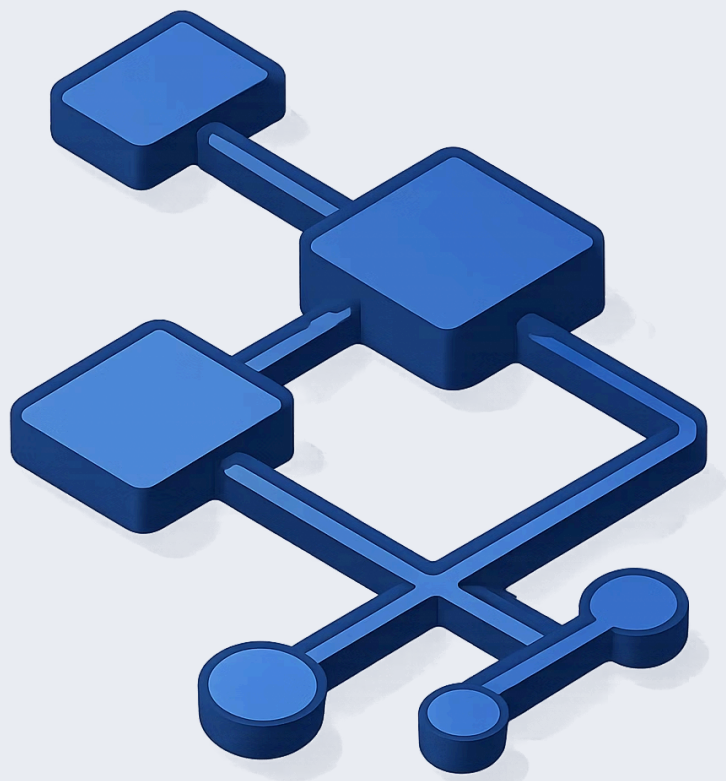
Phase 4: Evaluation & Persistence

- Generated Output Parsing
- Model Comparison (Base vs LLM vs Fine-Tuned+RAG)
- Save Results & Visualization (Metrics, Charts, Logs)

Pipeline Architecture Overview

End-to-End Pipeline (CodeGen Project)





Phase 1: System Setup & Configuration

01

Load Configuration

YAML settings define core parameters for model and dataset paths. This includes specifying local directories for pre-trained models and configuring the logging framework for structured output and debug trails. Finally, system paths are set, and GPU devices are assigned and configured.

02

Load Datasets

Benchmark datasets such as Spider, are loaded. A critical validation step ensures data integrity and format consistency, followed by preprocessing to prepare the data for efficient model consumption.

03

Base Model Preparation

The Qwen2.5-Coder-0.5B model is loaded, with INT8 quantization applied to reduce its memory footprint. This step also includes careful VRAM allocation and stability management to ensure optimal performance and resource utilization.

Phase 2: Core Task Pipelines

Advancing from baseline analysis, we detail the implementation of program and documentation generation, followed by comprehensive evaluation.



Program Generation

Generates functional scripts from specifications, using Qwen2.5-Coder-0.5B as the inference backbone. It supports syntax-aware decoding and multi-sample Pass@k evaluation, validated with execution-based metrics.



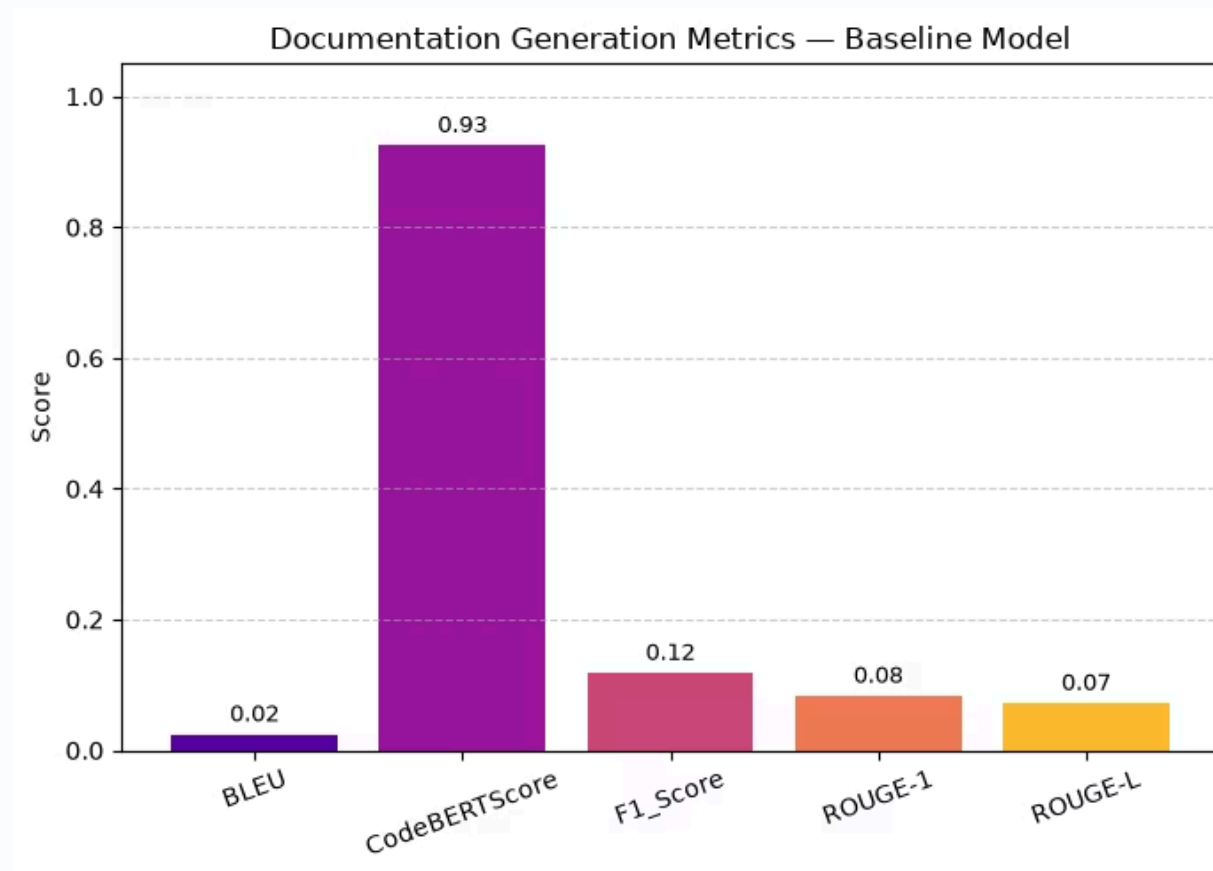
Documentation Generation

Creates docstrings and inline comments, producing module-level reference docs. This task summarizes inferred function semantics, defines interfaces, and parameter descriptions, translating analyzed logic into readable guidance.

Evaluation Metrics

- Pass@k: Execution accuracy
- BLEU: N-gram overlap
- BERTScore: Semantic similarity
- CodeBLEU: Code quality metric

Documentation Generation: Baseline Results



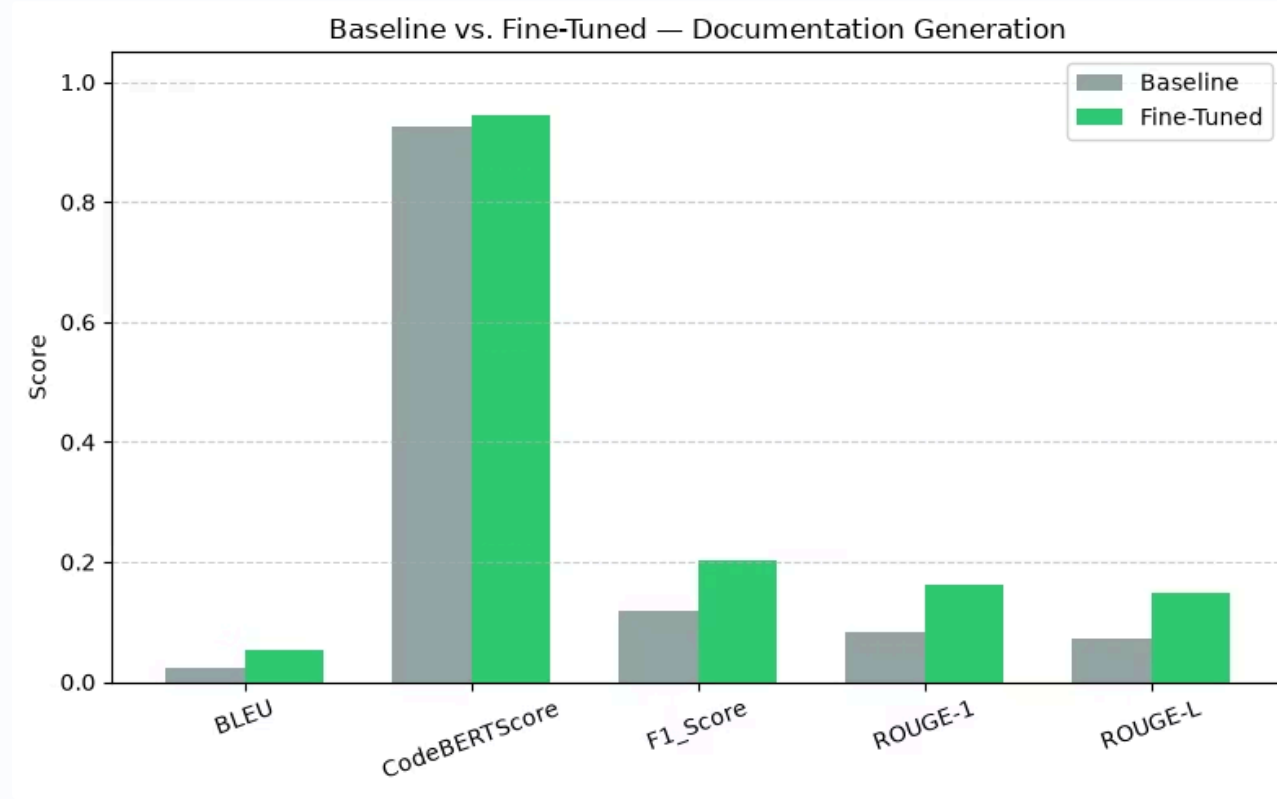
Baseline Observations

Out-of-the-box, the small code LM produces semantically coherent but poorly matched documentation.

- **BLEU:** 0.02 — near-zero n-gram overlap with reference docs
- **CodeBERTScore:** 0.93 — strong latent semantic alignment

The gap between BLEU and CodeBERTScore signals that the model captures meaning but not phrasing — a clear target for task-specific fine-tuning.

Baseline vs. Fine-Tuned: Documentation Generation



Significant Metric Uplift

Fine-tuning dramatically closes the gap between semantic understanding and surface-form accuracy.

- **CodeBERTScore:** 0.93 → 0.96 — stronger semantic fidelity to reference documentation
- Substantial gains in **BLEU** — the model now mirrors reference phrasing far more closely
- Improvements in **F1 Score** and **ROUGE** confirm better recall and precision across doc tokens

Task-specific adaptation transforms a semantically aware but stylistically misaligned baseline into a practical documentation assistant.



Phase 2: SQL & Commit Message Generation

Text-to-SQL Generation



- Schema resolution & multi-table joins
- Nested subquery support
- Multidimensional SQL benchmarks
- Execution accuracy validation

Commit Message Generation

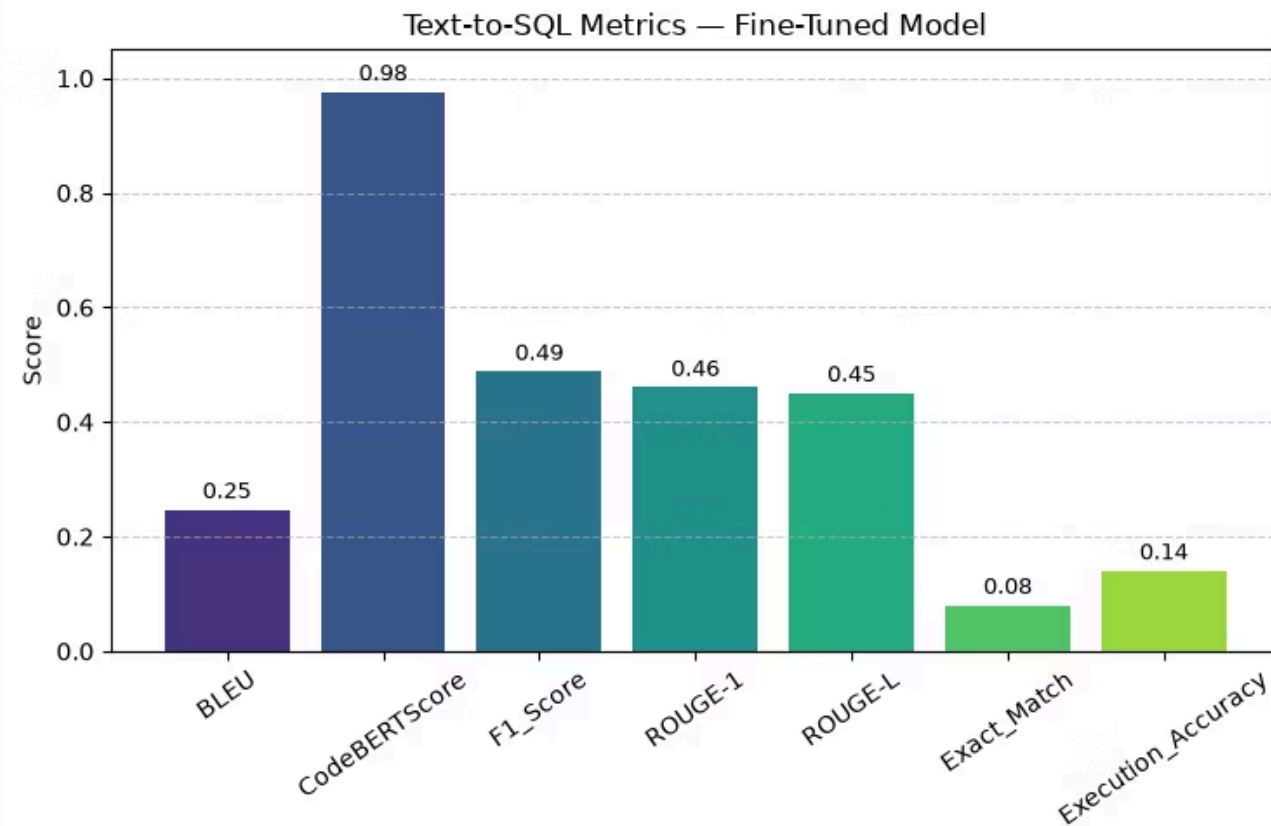


- Git diff parsing & semantic summarization
- Change intent alignment
- Structured code patch sets

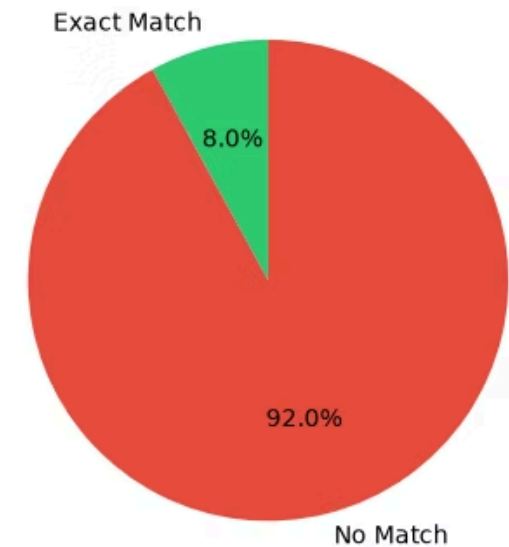
Evaluation Metrics

- **Execution Accuracy:** Validates generated SQL by executing against target database
- **Exact Match:** Checks structural equivalence of generated SQL
- **BLEU:** N-gram precision for commit message text alignment
- **ROUGE:** Recall-oriented overlap metric capturing coverage of reference content

Text-to-SQL: Baseline Model Performance



Gold vs. Predicted SQL — Exact Match Rate



Baseline Metrics at a Glance

The base model struggles with structural accuracy despite strong semantic similarity scores.

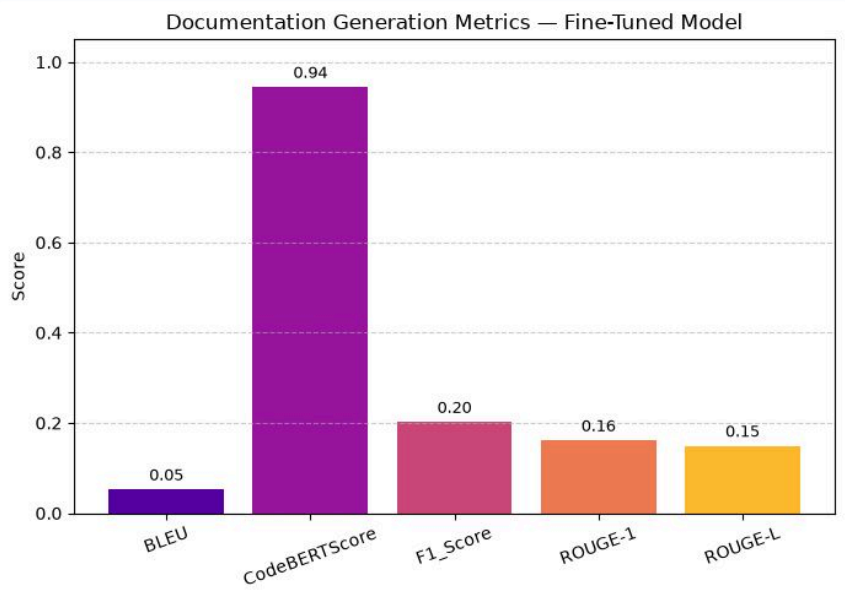
- **Exact Match:** 7% — very low structural fidelity to gold SQL queries
- **Execution Accuracy:** 11% — few generated queries produce correct results
- **CodeBERTScore:** 0.97 — high token-level semantic overlap
- **BLEU:** 0.23 — low n-gram precision highlights surface-form mismatch

Fine-Tuned Model Performance Comparison

Metric	Text-to-SQL (Fine-Tuned)	Documentation Generation (Fine-Tuned)
BLEU	0.2467	0.0528
CodeBERTScore	0.9759	0.9447
F1 Score	0.4879	0.2022
ROUGE-1	0.4602	0.1615
ROUGE-L	0.4503	0.1494
Exact Match	0.0800	N/A
Execution Accuracy	0.1400	N/A (free text)

Figure 4.3 — Text-to-SQL metrics and exact-match distribution, fine-tuned (LoRA) model, 100-sample subset.

Qualitative Samples: Text-to-SQL



Question	Gold SQL	Baseline Prediction	Fine-Tuned Prediction
How many singers do we have?	<code>SELECT count(*) FROM singer</code>	<code>SELECT count(*) FROM singer</code>	<code>SELECT count(*) FROM stadium</code>
Names, countries, ages for every singer, descending age order?	<code>SELECT name, country, age FROM singer ORDER BY age DESC</code>	<code>SELECT * FROM singer ORDER BY Age DESC</code>	<code>SELECT * FROM singer ORDER BY Age DESC</code>
Average, minimum, maximum age for all French singers?	<code>SELECT avg(age), min(age), max(age) FROM singer WHERE country = 'France'</code>	<code>SELECT AVG(age) FROM singer WHERE country = 'France'</code>	<code>SELECT AVG(age) FROM singer WHERE country = 'France'</code>
Name and release year of the song by the youngest singer?	<code>SELECT song_name, song_release_year FROM singer ORDER BY age LIMIT 1</code>	<code>SELECT * FROM singer WHERE Age = 25</code>	<code>SELECT * FROM singer WHERE Age = (SELECT MIN(Age) FROM singer)</code>
All countries and the number of singers in each country?	<code>SELECT country, count(*) FROM singer GROUP BY country</code>	<code>SELECT country FROM singer GROUP BY country</code>	<code>SELECT country FROM singer GROUP BY country</code>

Rows 1 and 4 illustrate semantic errors that persist—wrong table selection and alternative expressions of the same logic—both counted as mismatches by SQL Execution Accuracy.

Repository Structure

Path	Purpose
configs/config.yaml	Central YAML configuration (model names, LoRA hyperparameters, embedding source, retrieval/top-K sweep, LLM-baseline backend, generation settings, paths).
data/raw/Spider/	Raw Spider benchmark files (dev.json, tables.json, per-database SQLite files).
data/processed/	Normalized JSONL datasets: spider_eval.jsonl, retrieval_corpus.jsonl, spider_mongo_converted.jsonl, doc/program/commit_generation.jsonl.
data/indices/	Persisted FAISS index (faiss_semantic.index) and the AST/structural store (ast_store.pkl, 1,034 items), both actively used for hybrid RAG retrieval.
src/config.py	Typed dataclass configuration loader (paths, LoRA, embedding, RAG, generation, evaluation).
src/data/data_loader.py	Generic JSON/JSONL dataset loader (DatasetLoader).
src/embeddings/	CodeEmbedder (external BGE embeddings), CodeLMEmbedder (small-code-LM embeddings), SemanticIndexManager (FAISS), ASTIndexer (structural parsing for Python and SQL), and HybridRetriever (blended semantic + structural search).

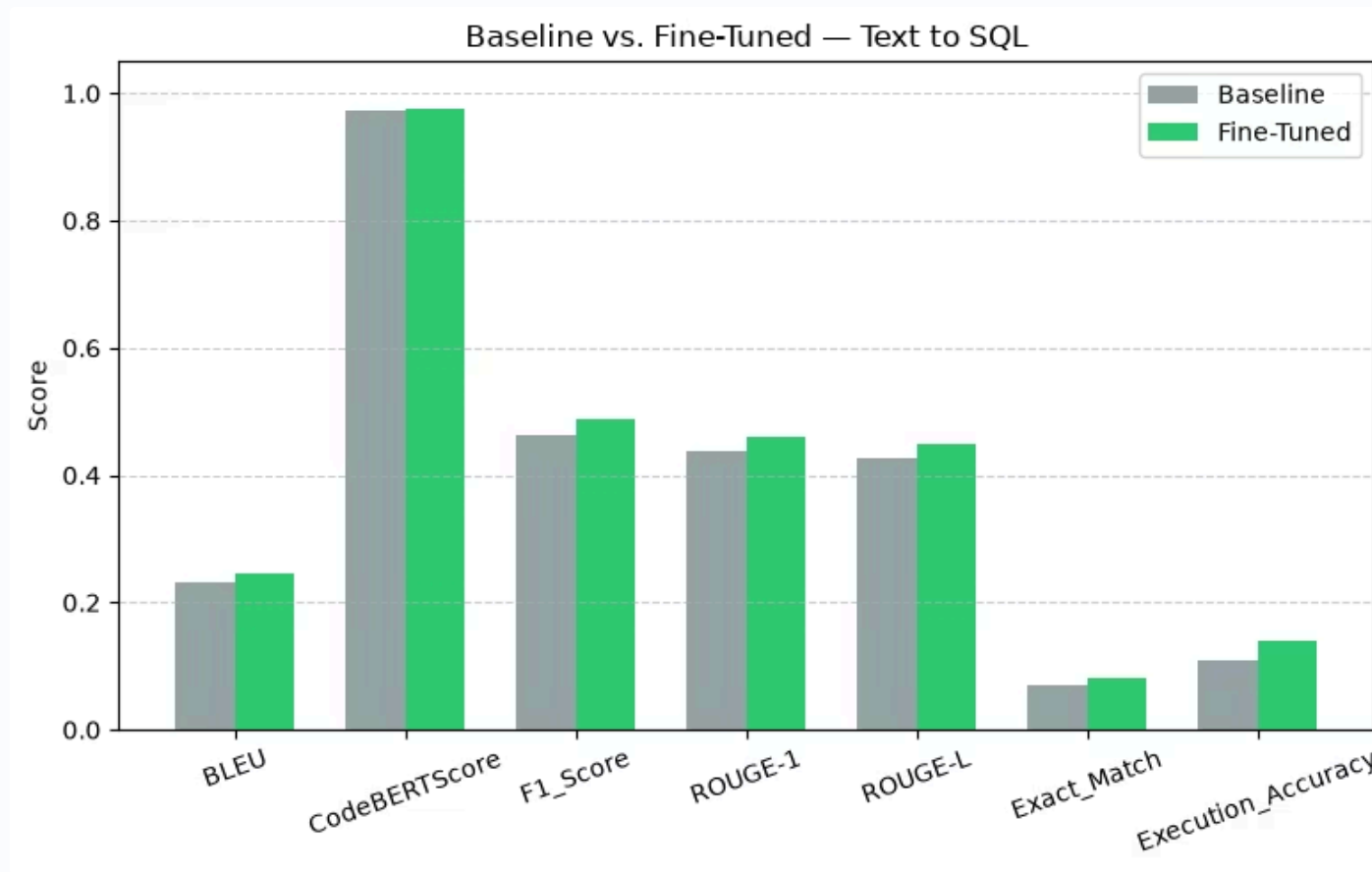
Repository Structure (Continued)

Path	Purpose
src/models/load_model.py	ModelLoader: base model/tokenizer loading, LoRA setup and training loop, base vs-LoRA generation.
src/generators/	Task-specific generation wrappers: program_generator.py, doc_gen.py, commit_gen.py, text_to_sql.py, sql_generator.py, and llm_baseline.py (LargeLLMGenerator, the LLM-comparison generator).
src/rag/rag_pipeline.py	RAGPipeline: retrieve_context(), build_prompt(), generate_with_rag() — backed by HybridRetriever.
src/evaluation/	metrics.py, evaluation.py, comparator.py, sql_eval.py, visualization.py, and rag_evaluation.py (RAGEvaluator — RAG-gain measurement and top-K sweep).
src/scripts/	convert_spider_to_jsonl.py, prepare_processed_data.py — one-off data preparation utilities.
output/plots/, src/plots/	Generated evaluation charts (per-model metrics, baseline-vs-fine-tuned comparisons, top-K gain sweep, full model comparison including the LLM baseline).
outputs/generated/	Saved generation samples and metric summaries (baseline_model_results.json, finetuned_model_results.json, per-task generated_samples.json, timestamped text-to-SQL runs).
models/checkpoints/lora_finetuned/	Saved LoRA adapter weights and tokenizer files.
main.ipynb	End-to-end Collab-oriented pipeline notebook (setup, fine-tuning, hybrid indexing, generation, evaluation, RAG-gain measurement, top-K sweep, and LLM-baseline comparison in one run).
baseline_model.ipynb	Loads only the pre-trained base model and evaluates it on text-to-SQL and documentation generation.
finetuned_model.ipynb	Loads/trains the LoRA adapter and evaluates the fine-tuned model on the same two tasks, then compares it to the baseline.
eval.ipynb	Full-scale (1,034-example) Spider evaluation of the example rule-based prediction file: exact-match and SQL execution accuracy.

Evaluation Metrics

Metric	What it measures	Applies to
BLEU	N-gram precision overlap between generated and reference text, with brevity penalty and additive smoothing.	SQL, code, docs
CodeBERTScore (cosine similarity)	Mean-pooled, L2-normalized microsoft/codebert-base embedding similarity between prediction and reference – more robust to paraphrasing/renaming than raw n-gram overlap.	SQL, code, docs
Token-level F1	Set-based precision/recall/F1 over whitespace-tokenized prediction vs. reference.	SQL, code, docs
ROUGE-1 / ROUGE-L	Unigram overlap and longest-common-subsequence overlap F-scores.	SQL, code, docs
Exact Match	Whitespace/case-normalized string equality between predicted and gold SQL.	SQL
SQL Execution Accuracy	Predicted and gold SQL are executed against the real Spider SQLite database and their result sets compared, so functionally equivalent (but textually different) queries are correctly credited.	SQL
Execution-based Pass Rate	Generated Python code is executed in a sandboxed subprocess against assertion-based test cases.	Program synthesis
Gain over no-RAG baseline	Per-metric difference (RAG or top-K result minus no-RAG baseline result), computed for the RAG-improvement and top-K sweep evaluation.	RAG evaluation

Baseline vs. Fine-Tuned: Text-to-SQL



Fine-Tuning Impact

LoRA adaptation delivers broad, consistent gains across every evaluation metric for Text-to-SQL.

- CodeBERTScore improves, reflecting better structural alignment with gold queries
- Execution accuracy rises — more generated SQL runs correctly against the database
- Consistent gains in **BLEU**, **F1 Score**, and **ROUGE** metrics

Phase 3: Semantic Indexing & RAG Engine

Semantic Vector Indexing

- Code Language Models convert code snippets into high-dimensional vector embeddings
- FAISS (Facebook AI Similarity Search) enables rapid retrieval of similar code snippets
- Fast similarity search quickly identifies semantically similar code
- Scalable to large codebases with millions of embeddings
- Semantic matching goes beyond keywords to understand code intent

Key Metrics

- **Similarity Scores:** Measures relevance between query and results
- **Retrieval Latency:** Time taken to fetch relevant code
- **Index Size:** Memory footprint of the FAISS index

Hybrid Retrieval System

Our novel hybrid retrieval system combines the best of semantic and structural matching to deliver highly relevant code snippets.



Vector Search (Semantic)

Leverages **FAISS** for efficient nearest-neighbor search, identifying conceptually similar code based on embedding similarity.



AST Structural Parsing

Utilizes **Tree-sitter** to parse code into Abstract Syntax Trees, enabling precise matching of structurally similar code patterns.



Fusion & Ranking

Results from both indices are weighted and combined using **reciprocal rank fusion (RRF)** for a refined set of candidates.

Phase 4: Fine-Tuning & Evaluation

This phase details the process of adapting our base model, integrating those adaptations, and rigorously assessing performance against established baselines.

LoRA Fine-Tuning

Applies lightweight LoRA adapters to update only a small set of parameters, preserving the base model while enabling efficient adaptation.

Adapter Merging

Merges adapter weights into the base model to produce a single, streamlined inference artifact, simplifying deployment-ready inference.

Benchmarking

Compares fine-tuned, baseline, and RAG-augmented models under the same conditions for fair comparison, measuring performance across key metrics.

Our Fine-Tuning Approach

01

Train efficiently using LoRA adapters

02

Merge for deployment into single model

03

Evaluate systematically with side-by-side comparisons

04

Score core tasks tracking code, text, and execution metrics

LoRA Fine-Tuning Configuration

16

LoRA Rank (r)

Alpha: 32 · Dropout: 0.05

0.436%

Trainable Params

2.16M of 496M total parameters updated

9

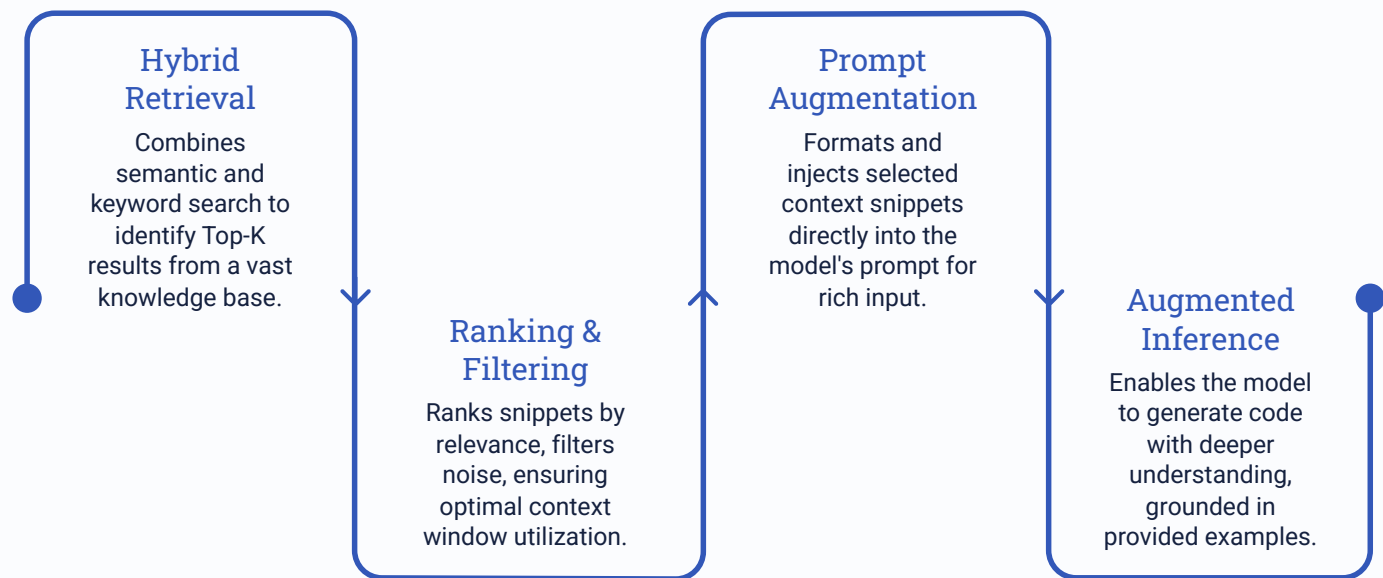
Training Pairs

Prompt/completion pairs from illustrative
task sets

Target modules: Attention projections — q_proj, k_proj, v_proj, o_proj

Context Injection Engine

Our Context Injection Engine optimizes language model performance by intelligently integrating relevant information into the generation process.



Key Benefits



Improved Accuracy

Provides directly relevant examples and facts to enhance generation precision.



Few-Shot Learning

Enables the model to learn and adapt quickly from specific patterns and examples.



Reduced Hallucinations

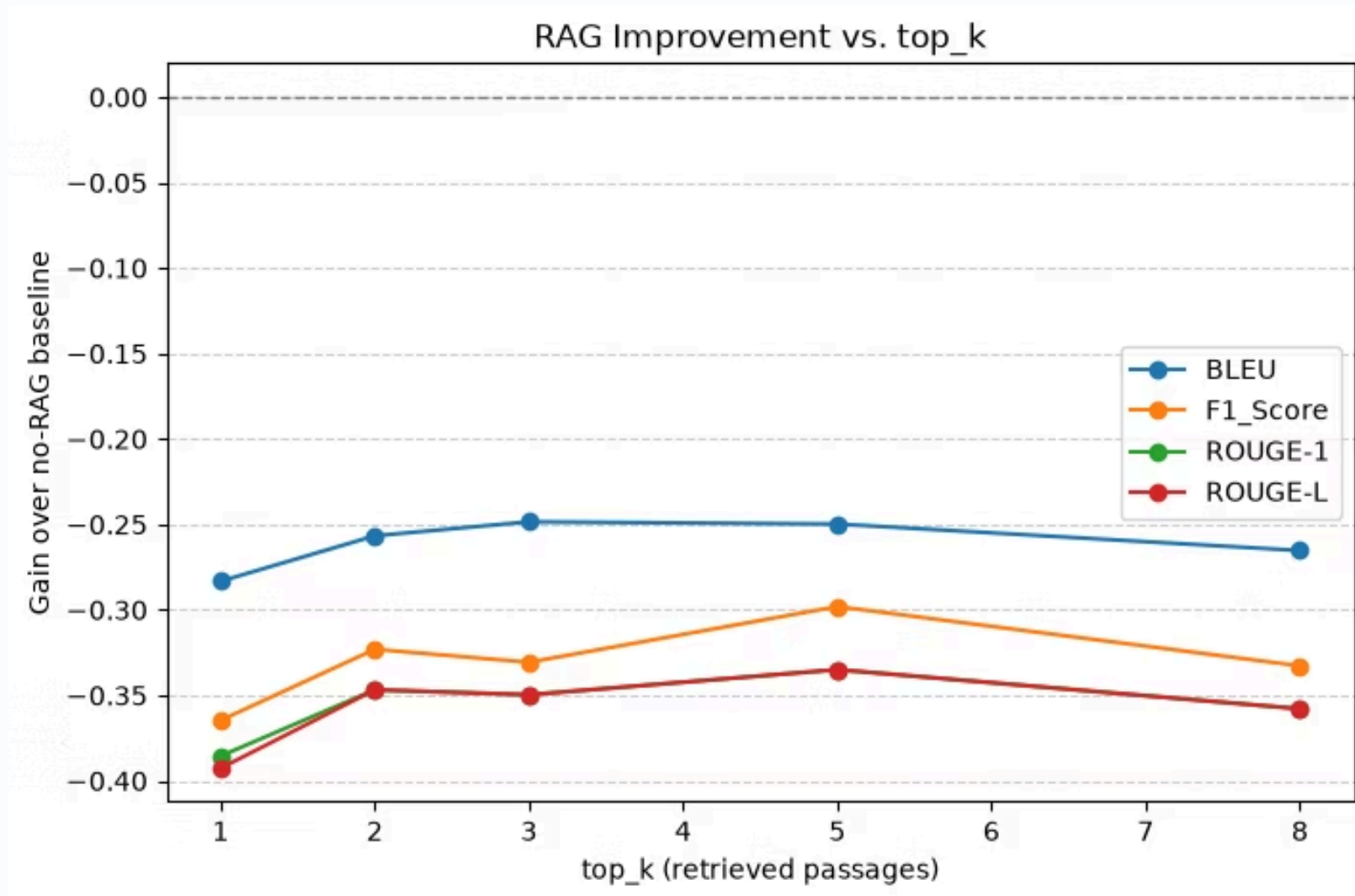
Grounds model output in real, verifiable information, minimizing erroneous generations.



Domain Adaptation

Tailors general language models to specific project codebases and unique programming environments.

RAG Improvement: Top-K Retrieval Sweep



Finding the Retrieval Sweet Spot

The top-K sweep reveals a clear performance peak — more retrieved context is not always better.

- **Optimal K = 3** for the majority of evaluation metrics across tasks
- Performance degrades at larger K values as noisy or irrelevant context dilutes generation quality
- Hybrid semantic + structural retrieval outperforms semantic-only indexing at every K level

Key Findings & Real-World Impact



Efficiency at Scale

Small code LMs match larger models with LoRA + RAG — dramatically reducing deployment costs and infrastructure overhead



Smarter Retrieval

Hybrid semantic-structural indexing consistently outperforms semantic-only approaches across all evaluated tasks



Rapid Adaptation

0.436% trainable parameters enable fast, low-cost task adaptation without full model retraining



Practical Applications

Interactive SQL querying, automated documentation, and code generation — ready for real developer workflows



Conclusion & Next Steps

Key Achievements

- Comprehensive end-to-end pipeline for multi-task code synthesis
- Hybrid indexing combining semantic and structural retrieval
- Parameter-efficient fine-tuning with LoRA adapters
- Systematic evaluation framework across multiple tasks

Real-World Applications

- Interactive SQL querying systems
- Automated documentation generation
- Code generation and completion
- Commit message automation
- Program synthesis from specifications

Future Enhancements

- Expand to additional programming languages
- Integrate with IDE plugins for real-time assistance
- Optimize retrieval latency for production deployment
- Explore multi-task learning approaches
- Implement continuous model improvement pipelines